

FortyGuard API Evaluation - Engineering Notes

Candidate: Sankalp Jha

Position: DevSecOps and Network Architect

Date: June 24, 2026

Repository: github.com/blackdragoon26/API-Eval

Scope: Temperature API, async job behavior, validation, credit accounting, docs accuracy, and a light dashboard/docs visual pass

Short Take

FortyGuard has the shape of a useful climate intelligence platform. The API is not just wrapping weather lookup; it is exposing heatmaps, satellite segmentation, street-view segmentation, environmental parameters, and report workflows around one async job model. That is the right direction for expensive geospatial work.

The final endpoint audit also corrected an early assumption: the Heatmap endpoint does work when called with the documented GeoJSON FeatureCollection payload. The stronger finding is more specific and more important: an intentionally unclosed polygon was also accepted, completed, and appeared in credit usage. That is the main trust issue I would fix first.

What I Tested

I ran a bounded review using the evaluation API key. I did not load test, bypass controls, or attempt destructive behavior. The key was passed through an environment variable and redacted from stored evidence.

Covered areas:

- Public readiness: /health, /ready, /openapi.json, /redoc
- Auth and validation basics
- Heatmap submission and status polling
- Satellite View Segmentation submission and status polling
- Street View Segmentation submission and status polling
- Environmental Parameters submission and status polling
- Credits usage
- Invalid activity lookup
- Documentation behavior and screenshots

Endpoint Results

Area	Observed result	Notes
GET /health	200	Health check works.
GET /ready	200	Readiness check works.
GET /openapi.json	200	OpenAPI 3.1 schema is available.
GET /docs	404	Swagger UI is disabled.
GET /redoc	200	ReDoc exists, but should be linked if intended for developers.
POST /v1/heatmap documented FeatureCollection	200 -> Completed	Returned map_data FeatureCollection and stats_data.
POST /v1/heatmap unclosed polygon	200 -> Completed	Invalid geometry was accepted and completed.

POST /v1/satellite valid request	200 -> Completed	Returned coordinates, image metadata, original_image, and segmentation.
POST /v1/streetview valid request	200 -> Completed	Returned front street-view segmentation output.
POST /v1/env_params OpenAPI request shape	200 -> Completed	Returned metadata, one location, environmental values, and solar irradiance.
POST /v1/heat_intelligence invalid category	422	Enum validation works.
GET /v1/status/not-a-real-id	403	Ambiguous for a nonexistent activity under a valid API key.
POST /v1/system/fetch-api-key-usage	200	Works, but docs do not present it cleanly as a normal API endpoint.

What Worked Well

The async `activity_id` model is a good fit. These are not simple CRUD calls; they are geo/computation jobs. Returning an ID and polling status is the right base design.

The endpoint set is coherent. Heatmap, Satellite, Street View, Environmental Parameters, and Heat Intelligence all point toward the same product thesis: operational temperature intelligence for cities and properties.

The OpenAPI schema is a strong asset. `/openapi.json` can support Postman imports, SDK generation, contract testing, and automated client validation if it is made more visible in the docs.

The credits endpoint returns useful plan and usage details. It just needs to be documented like the other APIs.

Main Findings

1. Invalid heatmap geometry is accepted and charged

The docs say polygons should be closed and invalid input should be rejected. In the final audit, I submitted an unclosed Heatmap `FeatureCollection`. The API returned 200, created an activity, completed the job, and the later credits usage showed two Heatmap Generation tasks for the run: the valid polygon and the unclosed polygon.

That should fail before enqueueing. A malformed polygon should return 400 or 422, with a clear error code, and should not consume credits.

Suggested fix:

- Validate ring closure, minimum points, self-intersection, area size, and supported geography before job creation.
- Return a stable error such as `invalid_polygon`.
- Do not deduct credits for validation failures.

2. Heatmap GeoJSON contract is easy to misread

The working example uses `FeatureCollection`, but parts of the docs say "GeoJSON polygon." Many developers would try sending a raw GeoJSON Polygon. Earlier probing showed that raw Polygon input can produce a 500, while the documented `FeatureCollection` shape works.

Suggested fix: make the accepted shape explicit in docs and OpenAPI. If only `FeatureCollection` is supported, reject raw Polygon with 422. If both are intended, normalize both safely.

3. Status and error semantics need tightening

GET /v1/status/not-a-real-id returned 403 `Unauthorized` access even though the request used a valid API key. That makes it hard for a client to know whether the ID is missing, belongs to another account, or the key is invalid.

Suggested fix: use 404 Not Found for nonexistent activities, or return a precise 403 only when the activity exists but belongs to another account.

4. Operational headers are missing

I did not observe headers for request correlation, rate limit state, retry timing, or remaining credits. For a credit-based async API, these headers are practical, not cosmetic.

Useful headers:

- X-Request-Id or X-Correlation-Id
- X-RateLimit-Limit
- X-RateLimit-Remaining
- Retry-After
- X-Credits-Remaining

5. Credits docs do not match the live API

The docs present credits usage like a GET/form-style page. The working API is:

```
POST /v1/system/fetch-api-key-usage
Body: {"api_key": "..."}

```

That endpoint is useful. It should have normal API documentation with request body, response schema, and examples.

6. Environmental Parameters docs and schema do not fully line up

The docs describe customizable environmental parameters, but the OpenAPI request schema does not expose a parameters selector. A request with a parameters array returned 422, while the OpenAPI-shaped request completed.

The result also needs clearer field docs for units and sentinel values, especially fields such as `air_quality:idx`, `air_quality_pm2p5:idx`, `cloud_cover_octas`, and values like `-999`.

7. Response naming inconsistency

Satellite results use `original_image`. Street View results use `original_image`. I would add the correctly spelled field, keep the old one temporarily for backward compatibility, and mark it deprecated.

Credit Accounting Observation

Before the final audit, used credits were 69,100. After the submitted jobs completed, used credits were 103,440, a delta of 34,340.

Activity	Added count	Added credits
Tile Satellite Segmentation	1	14,400
Streetview Segmentation	1	8,600
Heatmap Generation	2	8,440
Environment Parameter Analysis	1	2,900

The two heatmap tasks in that run were the valid `FeatureCollection` and the unclosed polygon. That supports the validation-before-billing finding.

Dashboard Notes

The dashboard and docs look modern and aligned with a map-based climate product. My concern is not visual polish; it is time-to-first-value. A new user should not have to guess the first useful action or how the dashboard result relates to an API job.

The dashboard would be stronger if each analysis made the job lifecycle visible: submitted input, validation result, current status, expected wait, credit impact, output summary, and export/API handoff.

Top Recommendations

- 1 Fix validation before billing. This is the highest-priority trust issue.
- 2 Make the API contract exact. Tighten OpenAPI, docs, examples, status codes, error envelopes, and field names.
- 3 Add production async features: webhooks, cancellation, idempotency keys, batch submission, and job history.
- 4 Add operational headers for request ID, rate limits, retry timing, and remaining credits.
- 5 Improve the dashboard handoff: export CSV/GeoJSON/PDF and show the equivalent API request for dashboard-generated work.

Suggested Ratings

Area	Rating	Reason
Dashboard usability	6/10	Product direction is clear, but first-run guidance and job visibility need work.
API	7/10	Main happy paths complete, but validation, billing safeguards, headers, and docs need tightening.
Overall experience	7/10	Strong foundation and good product thesis; most improvements are contract, trust, and workflow maturity.

Evidence

- Current API audit script: `scripts/final_api_endpoint_audit.py`
- Compatibility runner: `scripts/api_test_suite.py`
- Sanitized audit log: `evidence/api/final_api_endpoint_audit_2026-06-24.jsonl`
- Endpoint summary: `evidence/api/final_api_endpoint_summary_2026-06-24.md`
- Follow-up status checks: `evidence/api/final_status_followup_2026-06-24.jsonl`
- Credit usage after completion: `evidence/api/credits_usage_post_followup_2026-06-24.json`
- Visual/docs evidence: `evidence/visual/` and `evidence/docs/`

Final Takeaway

FortyGuard is close enough that the feedback can be concrete. The main API workflows work. The next step is to make the system feel dependable under real integration pressure: reject bad input early, do not charge for invalid jobs, expose operational signals, document the live contract accurately, and give developers better async controls.